

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1. Реализация REST API на основе
boilerplate

Выполнил:

Москалец Данила

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

LP1: Реализация REST API на основе boilerplate.

Необходимо реализовать модели, представления и контроллеры с учетом спроектированной базы данных и API. В результате выполнения работы требуется продемонстрировать полностью работающее API согласно выбранному варианту.

В проекте должны быть представлены сущности пользователей, компаний, категорий, услуг, скидок, заявок, отзывов и избранных услуг. Также необходимо реализовать авторизацию и разграничение прав по ролям.

Ход работы

1 Используемые технологии

Для реализации проекта использованы Node.js, Express, TypeScript, PostgreSQL и TypeORM. Валидация входящих данных выполняется с помощью Zod. Для защиты маршрутов используется JWT-аутентификация. Пароли пользователей хранятся не в открытом виде, а в виде bcrypt-хеша.

Swagger-документация подключена к приложению и доступна по адресу /api-docs. Это удобно для демонстрации работы API, потому что можно просматривать все маршруты, структуры запросов и ответов.

2 Структура приложения

Приложение разделено на модули по предметным областям:

- *auth* – регистрация, вход, refresh token, получение текущего пользователя;
- *user* – управление профилем;
- *company* – компании владельцев;
- *category* – категории услуг;
- *service* – услуги компаний и каталог;
- *discount* – скидки на услуги;
- *request* – заявки пользователей;
- *review* – отзывы;
- *favorite* – избранные услуги;

- *admin* – административные списки и отчеты.

Каждый модуль содержит контроллер, сервис, DTO и при необходимости TypeORM-сущность. Контроллер отвечает за HTTP-запросы и ответы, сервис содержит бизнес-логику, DTO описывает и валидирует входные данные, сущность описывает таблицу базы данных.

3 Работа авторизации и ролей

Пользователь регистрируется через */auth/register*, после чего может выполнить вход через */auth/login*. При успешном входе сервер возвращает пару токенов: access token и refresh token.

Access token передается в защищенные эндпоинты через заголовок:

Authorization: Bearer <access_token>

Роли используются для ограничения доступа:

- *USER* может просматривать каталог, создавать заявки, отзывы и избранное;
- *OWNER* может управлять своей компанией, услугами, скидками и заявками;
- *ADMIN* может управлять категориями, просматривать пользователей и отчеты.

4 Работа с заявками

Заявка создается пользователем на конкретную услугу. По умолчанию она получает статус *PENDING*. Дальше возможны только разрешенные переходы:

- *PENDING* -> *ACCEPTED*, если решение принимает владелец компании;
- *PENDING* -> *REJECTED*, если решение принимает владелец компании;
- *PENDING* -> *CANCELLED*, если заявку отменяет ее автор.

Из финальных статусов *ACCEPTED*, *REJECTED* и *CANCELLED* переходы запрещены. Такая логика реализует простую машину состояний и не позволяет менять заявку произвольно.

5 Работа с отзывами и избранным

Отзыв можно оставить только на услугу, по которой у пользователя есть принятая заявка. Это защищает систему от отзывов пользователей, которые фактически не пользовались услугой.

Избранное работает проще: пользователь может добавить услугу в список избранных и удалить ее оттуда. Повторное добавление той же услуги запрещено.

Вывод

В результате работы реализован REST API для маркетплейса охранных услуг. API поддерживает основные операции с пользователями, компаниями, услугами, заявками, отзывами, скидками и избранным. Также реализованы авторизация, роли, валидация входных данных, документация Swagger.